REFT: Reasoning with REinforced Fine-Tuning

Trung Quoc Luong*, **Xinbo Zhang***, **Zhanming Jie***, **Peng Sun**†, **Xiaoran Jin**, **Hang Li**
ByteDance Research
{trung.luong, zhangxinbo.freya, allan.jie}@bytedance.com
{wanhesong, xiaoran.jin, lihang.lh}@bytedance.com

arXiv:2401.08967v3 [cs.CL] 13 Dec 2024

Abstract

One way to enhance the reasoning capability of Large Language Models (LLMs) is to conduct Supervised Fine-Tuning (SFT) using Chain-of-Thought (CoT) annotations. This approach does not show sufficiently strong generalization ability, however, because the training only relies on the given CoT data. In math problem-solving, for example, there is usually only one annotated reasoning path for each question in the training data. Intuitively, it would be better for the algorithm to learn from multiple annotated reasoning paths given a question. To address this issue, we propose a simple yet effective approach called *Reinforced Fine-Tuning* (ReFT) to enhance the generalizability of learning LLMs for reasoning, with math problem-solving as an example. ReFT first warmups the model with SFT, and then employs on-line reinforcement learning, specifically the PPO algorithm in this paper, to further fine-tune the model, where an abundance of reasoning paths are automatically sampled given the question and the rewards are naturally derived from the ground-truth answers. Extensive experiments on GSM8K, MathQA, and SVAMP datasets show that ReFT significantly outperforms SFT, and the performance can be potentially further boosted by combining inference-time strategies such as majority voting and re-ranking. Note that ReFT obtains the improvement by learning from the same training questions as SFT, without relying on extra or augmented training questions. This indicates a superior generalization ability for ReFT [1].

1 Introduction

The state-of-the-art approaches to solving math problems (Luo et al., 2023; Wang et al., 2023a) employ Supervised Fine-Tuning (SFT) to train the models using Chain-of-Thought (CoT) annotations (Wei et al., 2022). As shown in Figure 1, a CoT annotation outlines the intermediate reasoning steps toward solving a math problem.

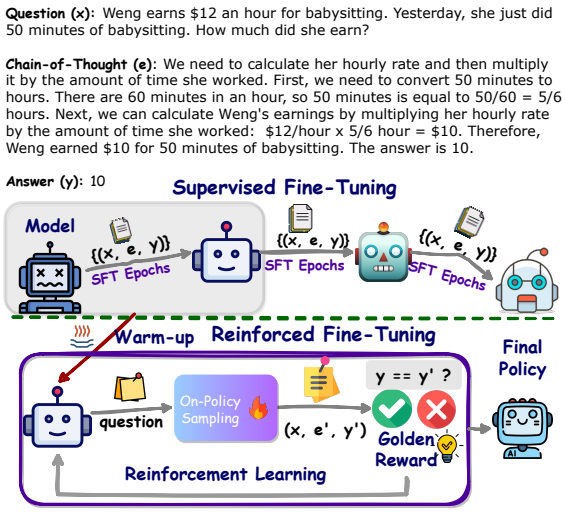


Figure 1: An example of question (x), CoT (e), and answer (y) in GSM8K (Cobbe et al., 2021a). The SFT process iterates several epochs on the training data. The proposed ReFT warm-up from SFT and performs RL training on the same data.

Usually there is one CoT annotation for each question in the training data, i.e., one correct reasoning path, which is utilized in SFT. We observe that this may result in relatively weak generalization abilities of the SFT models. It is often the case that multiple valid CoT annotations exist for the same question (Cobbe et al., 2021a; Zhang et al., 2023), underscoring the need for a more powerful fine-tuning approach. To address this problem, we propose a simple yet effective approach called *Reinforced Fine-Tuning* (ReFT) (Figure 1 bottom).

ReFT commences with a warm-up stage involving Supervised Fine-Tuning (SFT) in one or two epochs (Figure 1, shaded box). This initial stage equips the model with the ability to generate correct responses to mathematical problems to some extent, as demonstrated in prior work (Cobbe et al., 2021a). Next, ReFT proceeds to further refine the

* indicates equal contribution, † indicates corresponding author

[1] Code: https://github.com/lqtrung1998/mwp_ReFT

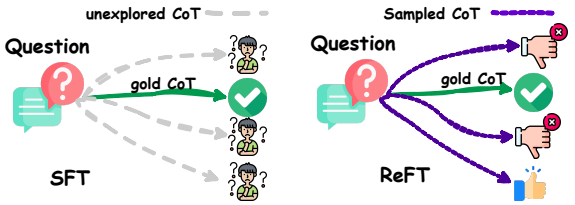


Figure 2: Comparison between SFT and ReFT on the presence of CoT alternatives.

model through the utilization of an online Reinforcement Learning (RL) algorithm (Sutton and Barto, 2018), specifically Proximal Policy Optimization (PPO) (Schulman et al., 2017) in this paper. In this way, ReFT is able to sample multiple correct reasoning paths or CoT annotations and learn from them (Figure 2, right).

Since the training data include ground-truth answers, the golden rewards can be naturally derived from them when training PPO. Consequently, there is no requirement for a separately trained reward model. In contrast, RLHF (Ouyang et al., 2022) has to utilize a reward model that is learned from human-labeled data.

During the warm-up stage, ReFT acquires a certain level of accuracy by supervised learning. In the RL stage, ReFT further enhances its ability by reinforcement learning through sampling various CoT reasoning paths. In this way, ReFT gets much richer supervision signals than SFT. This approach enables ReFT to greatly improve generalization in math problem-solving (Gao et al., 2018; Brown et al., 2020). Note that ReFT outperforms SFT by using the same training questions, without relying on extra or augmented training questions. In fact, ReFT does not conflict with such data engineering and can be seamlessly combined with it.

Our contributions are as follows:

- We introduce a novel fine-tuning approach, reinforced fine-tuning (ReFT), which utilizes reinforcement learning to solve math problems. ReFT exhibits enhanced generalization capabilities compared to conventional supervised fine-tuning when trained on the same dataset.
- We conduct extensive experiments using two foundational models, CodeLLAMA (Roziere et al., 2023) and Galactica (Taylor et al., 2022), on three standard datasets: GSM8K (Cobbe et al., 2021a), MathQA (Amini et al., 2019), and SVAMP (Patel et al., 2021). Our experiments cover both natural language and program-based CoTs, demonstrating the significantly improved performance and generalization ability of ReFT.
- Additionally, we demonstrate that ReFT benefits from both majority voting (Wang et al., 2023b) and reward model reranking (Uesato et al., 2022) at inference-time, further improving its performance.

2 Related Work

Math Problem Solving Recent research efforts focus on CoT prompt design and data engineering. Most of them attempted to make CoT comprehensive and fine-grained to present the step-by-step reasoning solutions (Nye et al., 2021; Fu et al., 2023; Zhou et al., 2023b; Khot et al., 2023; Zelikman et al., 2022; Imani et al., 2023; Miao et al., 2023). Gao et al. (2023) further proposed to use the Python program as CoT prompt, demonstrating more accurate reasoning steps and significant improvements over the natural language CoT (Wei et al., 2022). Zhou et al. (2023a) introduced a prompting method that generates code to verify the intermediate reasoning step with GPT-4 (OpenAI, 2023), thus achieving state-of-the-art performance on GSM8K (Cobbe et al., 2021a) and MATH (Hendrycks et al., 2021). Another line of work focuses on improving the quality of CoT (Wang et al., 2023a; Liu et al., 2023; Yu et al., 2023) and increasing the amount of CoT data (Luo et al., 2023; Yue et al., 2023) from OpenAI's ChatGPT (`gpt-3.5-turbo`) or GPT-4[2].

Reinforcement Learning Our work is mostly related to the recent work that applies PPO (Schulman et al., 2017) to natural language process for aligning human preferences (Ouyang et al., 2022). Since then, several training algorithms have been proposed to efficiently improve the alignment, including direct preference optimization (DPO) (Rafailov et al., 2023), identity preference optimization (IPO) (Azar et al., 2023), and Kahneman-Tversky optimization (KTO) (Ethayarajh et al., 2023). Other than the purpose of alignment, we aim to adopt reinforcement learning as a fine-tuning paradigm to improve performance over conventional supervised fine-tuning.

Specifically for solving math problems, Uesato et al. (2022) and Lightman et al. (2023) trained an outcome-based or process-based reward model to

[2] https://chat.openai.com/

perform reranking (Cobbe et al., 2021a) to achieve much better performance over SFT and majority voting (Wang et al., 2023b). While our approach aims to improve the performance of the policy itself, these reward model reranking approaches can be easily integrated into the resulting policy model.

3 Method

In this work, we focus on *natural language CoT* (**N-CoT**) (Wei et al., 2022) (Figure 1) and *program-based CoT* (Gao et al., 2023) (**P-CoT**) using Python. Gao et al. (2023) proposed the program-based CoT for math problem solving. We can simply execute the program to obtain the answer. To ensure clarity and avoid ambiguity, we use the terms N-CoT and P-CoT to represent natural language and program-based CoTs, respectively.

3.1 Reinforced Fine-Tuning

The proposed Reinforced Fine-Tuning (ReFT) process consists of two stages: the warm-up stage and the reinforcement learning stage. The overall algorithm is shown in Algorithm 1.

Warm-up In this stage, the policy is fine-tuned for a few epochs on a dataset comprising of the "(*question*, *CoT*)" tuples: $(\boldsymbol{x}, \boldsymbol{e})$. It enables the model to have basic problem-solving skills to generate a proper response[3]. Formally, the CoT generation process can be decomposed into a sequence of next token prediction actions. The last action token, <eos>, signals the generation process to terminate. The CoT $\boldsymbol{e}$ is written as:

$$\boldsymbol{e} = [a_1, a_2, ..., a_{L-1}, a_L\texttt{=<eos>}]$$

where L represents the maximum length. At timestep t, the action a_t is sampled from a policy $\boldsymbol{\pi_\theta}(\cdot|s_t)$ where a_t can be any token in the vocabulary and the state s_t comprises of all tokens in the question and all tokens generated so far. After each action, the resulting state s_{t+1} is the concatenation of the current state s_t and the action a_t:

$$s_{t+1} = \begin{cases} \boldsymbol{x}, & t = 0 \\ [s_t, a_t], & 1 \leq t \leq L \end{cases}.$$

As the produced action is the <eos> token, the resulting state s_{L+1} is the terminal state and the generation process is finished. With this notation, the loss function for a sample can be written as:

$$\mathcal{L}_{SFT}(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{e}\sim\mathcal{D}}\left[\sum_{t=1}^{L} \log\left(\boldsymbol{\pi_\theta}(a_t|s_t)\right)\right] \quad (1)$$

Reinforcement Learning In this stage, the policy improves its performance via a form of online self-learning using a dataset comprising of (*question*, *answer*) tuples: $(\boldsymbol{x}, \boldsymbol{y})$. Specifically, the policy model learns by repeatedly sampling responses (Figure 2), evaluating the response's answer correctness, and updating its parameters in an online fashion (line 7-14 in Algorithm 1). We employ PPO (Schulman et al., 2017) with a clipped objective algorithm for training. Following Ziegler et al. (2019), the value model V_ϕ is constructed by appending a linear value head on top of the last hidden states of the policy model π_θ, which is the model after the warm-up stage. The reward of 0 is given for all action resulting in non-terminal state. At the terminal state, we use a reward function that directly compares the answer extracted from the state's CoT and the ground-truth answer $\boldsymbol{y}$. Here, the reward function returns 1 if the answer is deemed correct, otherwise 0 is returned. On dataset whose answers are all numeric, *partial reward* (Zhong et al., 2017; Le et al., 2022) of 0.1 can be applied when the answer can be extracted and it is of numeric type. For $1 \leq t \leq L$, we write

$$r(s_t, a_t, s_{t+1}) = \begin{cases} 1, & \texttt{EXTRACT}(s_{t+1}) = \boldsymbol{y} \\ 0.1, & \texttt{EXTRACT}(s_{t+1}) \neq \texttt{null}, \neq \boldsymbol{y} \\ 0, & \texttt{EXTRACT}(s_{t+1}) = \texttt{null} \end{cases}$$

Such a partial reward can help reduce the effect of learning from sparse reward (Riedmiller et al., 2018; Trott et al., 2019). In addition, following Zheng et al. (2023), our total reward is the sum of the reward function score and the Kullback-Leibler (KL) divergence (Kullback and Leibler, 1951) between the learned RL policy and initial policy scaled by a coefficient factor β.

$$r_{total}(s_t, a_t, s_{t+1}) = r(s_t, a_t, s_{t+1}) - \beta KL\left(\boldsymbol{\pi_\theta}(\cdot|s_t), \boldsymbol{\pi_\theta^{(0)}}(\cdot|s_t)\right)$$

The generalized advantage estimate (Schulman et al., 2018) is used for advantage calculation:

$$\hat{A}_t = \sum_{l=0}^{L-t} (\gamma\lambda)^l \delta_{t+l},$$

[3]The underlying concept is similar to the verifier training (Cobbe et al., 2021a) to generate multiple solutions.

Algorithm 1: Reinforced Fine-Tuning

Input: $\mathcal{D}_{train} = \{(\boldsymbol{x}, \boldsymbol{e}, \boldsymbol{y})\}$: Tuples of (*question*, *CoT*, *answer*), W: number of warm-up steps, T: number of RL steps, U: number of updates per RL step, $\boldsymbol{\pi}_{\theta}^{(0)}$: Initial policy.

Output: $\boldsymbol{\pi}_{\boldsymbol{\theta}}$: Final policy

```
π_θ = π_θ^(0)
// Warm-up stage
for i ← 1 to W do
    x, e, y ~ D_train                                   // Sample mini-batch from D_train
    θ = OPTIMIZATION_STEP(L_SFT(θ))                     // Equation 1
// Reinforcement learning stage
for i ← 1 to T do
    x, _, y ~ D_train                                   // Sample mini-batch without CoT
    ê ~ π_θ(x)                                          // On-policy CoT sampling
    ŷ ← EXTRACT(ê)                                      // Extract the answer from CoT
    π_θ_old ← π_θ, V_φ_old ← V_φ
    Compute δ_t, Â_t, R̂_t using π_θ_old, V_φ_old, x, ê, ŷ and y
    for j ← 1 to U do
        θ, φ = OPTIMIZATION_STEP(L_RL(θ, φ))            // Equation 2
return π_θ
```

where the Temporal Difference (TD) is defined as

$$\delta_{t'} = -V_{\phi}(s_{t'}) + r_{total}(s_{t'}, a_{t'}, s_{t'+1}) + \gamma V_{\phi}(s_{t'+1})$$

with the terminal state value $V_{\phi}(s_{L+1}) := 0$, $\lambda \in (0, 1]$ is the discount factor for rewards, and $\gamma \in [0, 1]$ is the discount factor for TD. For the estimate of return, we leverages the λ-return $\hat{R}_t$, which can be written as the sum of the generalized advantage estimate and the value estimate:

$$\hat{R}_t = \hat{A}_t + V_{\boldsymbol{\phi}}(s_t)$$

Lastly, the policy and value objectives can be written as in two equations below

$$\mathcal{L}_{policy}(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{e} \sim \boldsymbol{\pi}_{\theta_{\text{old}}}} \left[\min \left(\frac{\boldsymbol{\pi}_{\boldsymbol{\theta}}(a_t|s_t)}{\boldsymbol{\pi}_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t, \right. \right.$$
$$\left. \left. \text{clip} \left(\frac{\boldsymbol{\pi}_{\boldsymbol{\theta}}(a_t|s_t)}{\boldsymbol{\pi}_{\theta_{\text{old}}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

$$\mathcal{L}_{value}(\boldsymbol{\phi}) = \frac{1}{2} \mathbb{E}_{\boldsymbol{e} \sim \boldsymbol{\pi}_{\theta_{\text{old}}}} \left[\max \left(\left\| V_{\boldsymbol{\phi}}(s_t) - \hat{R}_t \right\|^2, \right. \right.$$
$$\left. \left. \left\| \text{clip} \left(\hat{R}_t - V_{\phi}(s_t), \hat{A}_t - \epsilon, \hat{A}_t + \epsilon \right) \right\|^2 \right) \right]$$

where $\boldsymbol{\pi}_{\theta_{\text{old}}}$, $V_{\phi_{\text{old}}}$ are used for sampling CoT and computing $\hat{A}_t$, $\hat{R}_t$. The unified loss function is the weighted sum of the above objectives.

$$\mathcal{L}_{RL}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathcal{L}_{policy} + \alpha \mathcal{L}_{value} \tag{2}$$

where α is the coefficient for the value objective.

4 Experiments

4.1 Datasets

We conduct experiments on three math problem datasets: GSM8K (Cobbe et al., 2021a), SVAMP (Patel et al., 2021) and MathQA (Amini et al., 2019). For both GSM8K and SVAMP, the format of answers is a numeric value. In MathQA, the format is instead a list of multiple choices (i.e., `ABCD`). Table 1 presents the statistics of all datasets. We perform few-shot prompting (Wei et al., 2022; Gao et al., 2023) using `GPT-3.5-turbo` to obtain both the N-CoT and P-CoT annotations[4]. The N-CoT and P-CoT annotations are obtained following Jie et al. (2023). We also conducted an additional experiment on a numeric version of MathQA (Jie and Lu, 2023) where the format is also a numeric value. Such experiments are used to demonstrate our assumptions of potential reward hacking phenomenon (Skalse et al., 2022) on MathQA (§4.4).

4.2 Baseline

We compare ReFT with SFT and self-training (Xie et al., 2020; Amini et al., 2022) baselines. SFT simply fine-tunes the language model on the train-

[4]Examples of N-CoT and P-CoT representations can be found in Appendix A.

	GSM8k	SVAMP	$\text{MathQA}_{\text{MCQ}}$	$\text{MathQA}_{\text{numeric}}$
Train N-CoT	7,465	3,076	14,862	8,955
Train P-CoT	7,356	3,043	15,250	7,672
Test	1,319	1,000	1,605	1,605

Table 1: Statistics of the train and test datasets.

ing data. Experiments with self-training methods ensure a relatively fair comparison because these methods share the mechanism that the samples generated from the model are used for training.

We implemented Offline Self-Training (**Offline-ST**) (He et al., 2020), and Online (Hoi et al., 2021) Self-Training (**Online-ST**). The Offline-ST method is similar to expert iteration (Anthony et al., 2017; Uesato et al., 2022; Zelikman et al., 2022). We first use the SFT checkpoint from the early checkpoint to sample the CoTs and verify them against the ground truth. We only retain those expert samples that have a correct answer. We perform SFT on the combination of original training data and the expert samples.

The Online-ST method is made to be closely comparable to ReFT. Following ReFT, Online-ST has the same warm-up process. After that, we perform continual training with the samples generated on the fly. At each training step, the model first samples CoTs for a batch and only retains those with correct answers. The resulting batch consists of both sampled and ground-truth CoTs. We then update the model parameters on this batch with the supervised fine-tuning objective $\mathcal{L}_{SFT}$. Compared with ReFT, Online-ST neither makes use of negative responses (with an incorrect answer) nor has a dedicated mechanism to prevent the model from significantly diverging from the initial model, which can manifest as task-specific overfitting and training instability.

4.3 Experimental Setup

We conduct experiments with two foundation models: Galactica-6.7B[5] (Taylor et al., 2022) and CodeLLAMA-7B[6][7] (Roziere et al., 2023). Both models are reported to have strong performance in math solving and are commonly adopted in recent literature on reasoning tasks (Yue et al., 2023; Luo et al., 2023).

In addition to the comparison with baselines, we also apply common techniques, majority voting (Wang et al., 2023b) and reward model reranking (Lightman et al., 2023) on GSM8K.

Hyper-parameters In all experiments, the training is done with 8 A100-80GB GPUs using DeepSpeed (Rajbhandari et al., 2020; Rasley et al., 2020) Zero stage 2 and HuggingFace Accelerate (Gugger et al., 2022). During the warm-up stage of ReFT, we use AdamW (Loshchilov and Hutter, 2017) optimizer with 10% warm-up ratio. The batch size is 48 and learning rate is $1e$-5. The maximum length is set to 1024. The number of epochs in the warm-up stage is 2 in all settings except on $\text{MathQA}_{\text{MCQ}}$ and $\text{MathQA}_{\text{numeric}}$ where we use up to 5 and 10 respectively. The model is trained for 300 epochs with a learning rate of $3e$-7. Following Ziegler et al. (2019), the λ, γ, α, ϵ and U in PPO are set to 1, 0.95, 5, 0.2, and 2, respectively. The KL coefficient β is set to 0.01 for P-CoT and is set to 0.05 for N-CoT experiments. Further hyperprameter settings about ReFT can be found in Appendix B.

For SFT baseline, we train the model for 40 epochs and choose the checkpoint with best performance. This number of epochs has been chosen to be sufficiently large to ensure SFT converges. For Offline-ST baseline, we sample the CoTs by using the checkpoint from the ReFT warm-up stage. Using the generation temperature of 1.0 and max length of 1024, we sample 100 CoTs for each question and only keep those with a correct answer. Following Singh et al. (2023), we then subsample the CoTs to 10 random unique CoTs per question to balance difficulties of questions. The number of fine-tune epoch is set to 20, which is sufficiently large to ensure the training to converge. As mentioned in §4.2, the Online-ST baseline tries to mimic the same setting as in ReFT. We have the same warm-up process and the hyperparameter setting is roughly the same as ReFT.

Reward Model Reranking Following (Cobbe et al., 2021a; Uesato et al., 2022), we train a reward model (RM) to determine the correctness of the CoT. To construct the RM training data, we use the model from the warm-up stage and perform sampling to obtain 100 CoTs for each question in the training set. The CoTs are deduplicated and the binary labels can be obtained by comparing the extracted answer against the ground truth.

As a common practice, the reward model is a

[5] huggingface.co/facebook/galactica-6.7b

[6] huggingface.co/codellama/CodeLlama-7b-hf

[7] Additional preliminary experiments were conducted using Gemma (GemmaTeam, 2024). However, these results are not included in the current version of this paper due to unresolved implementation issues that align with known challenges reported within the open-source community (https://huggingface.co/google/gemma-7b/discussions).

Method	Size	GSM8K		SVAMP		$\text{MathQA}_{\text{MCQ}}$		Average	
		N-CoT	P-CoT	N-CoT	P-CoT	N-CoT	P-CoT	N-CoT	P-CoT
Galactica + SFT	6.7B	42.68	58.83	54.50	70.09	58.07	64.61	51.75	64.51
Galactica + Offline Self-Training	6.7B	42.60	60.72	57.90	72.30	**60.75**	67.04	53.75	66.69
Galactica + Online Self-Training	6.7B	47.84	62.93	59.40	**74.59**	59.38	61.24	55.54	66.25
Galactica + ReFT	6.7B	**48.14**	**68.91**	**61.40**	74.09	58.13	**70.47**	**55.89**	**71.16**
CodeLLAMA + SFT	7B	43.59	63.68	58.09	75.40	56.01	64.79	52.56	67.96
CodeLLAMA + Offline Self-Training	7B	45.10	68.00	60.20	77.69	59.81	68.53	55.04	71.41
CodeLLAMA + Online Self-Training	7B	44.66	67.85	58.60	77.40	56.95	68.85	53.40	71.37
CodeLLAMA + ReFT	7B	**53.30**	**75.28**	**64.50**	**79.19**	**60.13**	**71.83**	**59.31**	**75.43**

Table 2: Value accuracy of ReFT and the baselines fine-tuned with two foundation models on all datasets.

language model that is initialized from the best SFT checkpoint (Cobbe et al., 2021a; Ouyang et al., 2022). Similar to the outcome-based reward model (ORM) (Uesato et al., 2022), the reward model is trained to predict a binary label that indicates the "*correct*" or "*incorrect*" solution. Once the input passes through the reward model, classification is conducted with a linear classifier on the hidden state of the last token. Finally, the solution with the highest "correct" score among the candidates is selected as the final answer. We train the RM model for 3 epochs using a batch size of 24, the maximum length of 700 and a linear learning rate schedule with 10% warm-up period and the max learning rate of 1e−6.

Evaluation We report value accuracy for both N-CoT and P-CoT on all datasets. For majority voting and reranking (Table 4), we sample 100 CoTs for evaluation. In voting, the valid answer with majority counts is chosen as the final answer for computing accuracy. In reranking, we choose the CoT with the highest score and extract the answer.

4.4 Results

ReFT Outperforms SFT Table 2 compares the performance among the baselines and proposed ReFT on GSM8K, SVAMP, and MathQA datasets. We can observe that ReFT consistently achieves much better performance over the SFT except on $\text{MathQA}_{\text{MCQ}}$ N-CoT. Specifically, we have closed to 10-point and 12-point improvement over SFT with CodeLLAMA on GSM8K N-CoT and P-CoT, respectively. On average, we achieve 6.7-point and 7.4-point improvements with CodeLLAMA on all datasets in N-CoT and P-CoT, respectively. Notably, no additional annotations or reward models are used in ReFT. Such strong results demonstrate robust generalization of ReFT (see Analysis §5) and huge potential for further exploring the training data with reinforcement learning (Lu et al., 2023).

Question: The diagonals of a rhombus are 18 cm and 22 cm. Find its area?
A) 277, B) 266, **C) 198**, D) 288, E) 212

Generated CoT: To find the area of a rhombus, we need to multiply the lengths of its diagonals and divide the result by 2.
Area of rhombus = (Product of diagonals) / 2
Area of rhombus = (18 cm x 22 cm) / 2
Area of rhombus = 344 cm^2 / 2
Area of rhombus = 172 cm^2
Therefore, the answer is: C

Figure 3: Example prediction of $\text{MathQA}_{\text{MCQ}}$ reveals reward hacking.

N-CoT	Galactica	CodeLLAMA
SFT	40.08	37.32
Offline Seft-Training	44.23	41.24
Online Seft-Training	43.78	38.06
ReFT	**45.23**	**42.24**

Table 3: Value accuracy of ReFT and the baselines with two foundation models on $\text{MathQA}_{\text{numeric}}$ benchmark

Offline self-training includes the sampling data from the initial policy for fine-tuning. We can see this simple baseline can improve the performance compared with SFT (He et al., 2020; Gulcehre et al., 2023) but the improvements are far behind the one made by ReFT. Such comparisons indicate that "*exploring*" is essential in ReFT to have good performance. Though online self-training achieves some more improvements with Galactica, it is still far behind ReFT on average. This result indicates that incorrect instances are also very essential to guide the model for better exploration. Comparisons with self-training also suggest the proposed approach with on-policy sampling and reinforcement learning is better than standard data augmentation approaches.

Reward Hacking for MathQA Our investigation of the negative results on $\text{MathQA}_{\text{MCQ}}$ in-

Method	Size	GSM8K		Extra SFT Data
		N-CoT	P-CoT	
Galactica + SFT + Voting	6.7B	52.8	62.9	✗
Galactica + ReFT + Voting	6.7B	58.5	71.8	✗
Galactica + SFT + Reranking	6.7B	57.5	73.4	✗
Galactica + ReFT + Reranking	6.7B	**59.2**	**76.4**	✗
CodeLLAMA + SFT + Voting	7B	53.5	68.0	✗
CodeLLAMA + ReFT + Voting	7B	63.2	78.0	✗
CodeLLAMA + SFT + Reranking	7B	62.9	77.0	✗
CodeLLAMA + ReFT + Reranking	7B	**66.0**	**81.2**	✗
Other Foundation Models †				
WizardMath (Luo et al., 2023)	7B	54.9	-	✓ (96k)
WizardMath (Luo et al., 2023)	13B	63.9	-	✓ (96k)
MathCoder (Wang et al., 2023a)	7B	67.8	-	✓ (80k)
MAmmoTH-Coder (Yue et al., 2023)	7B	22.2	58.8	✓ (260k)
MAmmoTH-Coder (Yue et al., 2023)	70B	72.4	76.7	✓ (260k)
DeepSeekMath (Shao et al., 2024)	7B	88.2	86.7	✓ (776k)
GPT-3.5-turbo (Jie et al., 2023)	N.A.	75.3	78.0	N.A.
GPT-4 (OpenAI, 2023; Zhou et al., 2023a)	N.A.	93.0	97.0	N.A.

Table 4: Solving accuracy of majority voting and reward model reranking for SFT and ReFT on GSM8K. We also include existing approaches for comparison.

dicates that ReFT suffers from the reward hacking (Skalse et al., 2022) on the multi-choice question during training. Figure 3 shows how the sampled solutions produce "*inaccurate rewards*", which makes the RL training suffer. As we can see, the sampled CoT obtains an incorrect answer "*172*" which is not half of the product of "*18*" and "*22*". However, the final reasoning step still predicts the option "*C*" as the final answer as the model would always predict one of the options from {`A, B, C, D, E`} regardless of the correctness of intermediate CoT[8]. Thus, such a misleading CoT will receive a positive reward "1" and misguide the model to treat this as a correct CoT. The underlying reward hacking phenomenon severely tampers the model training (Everitt et al., 2021). This is also the reason that we chose the checkpoint with longer warm-up steps for MathQA N-CoT to reduce the reward hacking effect.

To further demonstrate the negative effect of MCQ questions, we experiment on the MathQA variant by Jie and Lu (2023), $\text{MathQA}_{\text{numeric}}$ (Table 1), which removes the options in the question, and directly predicts the numeric answer. Table 3 presents the comparison against the baselines. We can observe that ReFT consistently outperforms the baselines using both Galactica and CodeLLAMA. Ideally, we could reduce the reward hacking effect on $\text{MathQA}_{\text{MCQ}}$ if we can obtain a more fine-grained reward (e.g., process-based reward (Lightman et al., 2023)) for the intermediate reasoning steps. However, the development of a reliable process-based reward model is expensive, and requires extensive manual annotations of reasoning steps. Recognizing these challenges, we consider controlling reward hacking and its analysis as an important problem to be addressed in future work.

[8]We found that program-based CoTs are less likely to suffer as it is more rigorous than natural language.

Method	GSM8K	SVAMP	$\text{MathQA}_{\text{MCQ}}$
Galactica-125M + SFT	23.7	35.6	58.4
Galactica-125M + ReFT	29.8	39.4	60.7
Codeparrot-small + SFT	13.8	25.7	55.3
Codeparrot-small + ReFT	16.8	27.4	58.3
Codegen-350M + SFT	20.4	34.4	56.4
Codegen-350M + ReFT	28.4	39.3	59.1

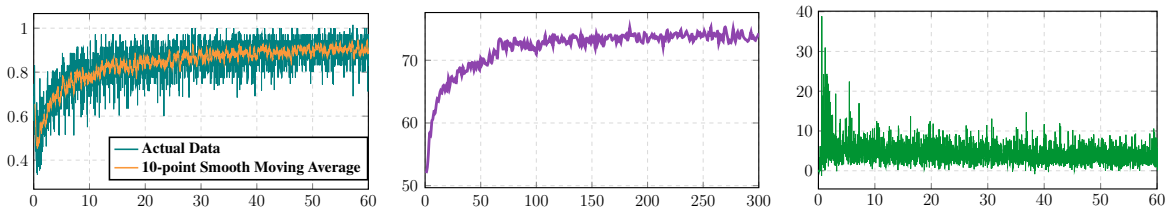
Table 5: Experiments on P-CoT with Galactica-125M, Codeparrot-small and Codegen-350M.

Majority Voting and Reranking Benefit ReFT Following Wang et al. (2023b); Uesato et al. (2022); Lightman et al. (2023), we also perform majority voting and reward model reranking to show that ReFT can benefit from these common techniques. Specifically, we perform sampling from both SFT and ReFT policies. We sample 100 CoT solutions for each question and employ the reward model described in §4.3 to perform reranking. Results in Table 4 demonstrate that ReFT consistently achieves the best performance on GSM8K by reward model reranking. ReFT + Voting significantly outperforms SFT + Voting by 8.6 points on average across all settings. ReFT with reranking outperforms SFT with reranking by more than 3 points.

Compared with existing open-source approaches (Luo et al., 2023; Wang et al., 2023a; Yue et al., 2023) (Table 4 bottom[9]), our best P-CoT variant achieves the best performance with accuracy 81.2 on GSM8K. In addition, these approaches mainly include extra data generated from ChatGPT and perform distillation during fine-tuning. In contrast, we improve the policy itself by exploiting the potential of existing training data and pushing the limit of the policy performance. Our best result reported in Table 4, i.e., the CodeLLAMA + ReFT + Reranking with P-CoT setting, even surpasses `GPT-3.5-turbo`. However, we obtain the result with a model that is only in the size of 7B.

Experiments with Small Model Intuitively, exploration could lead to imperfect demonstration

[9]Numbers are taken from original papers. The N-CoT and P-CoT results for MAmmoTH-Coder are reported in their appendix.



(a) Mean Training reward (b) Evaluation accuracy (c) Mean Sequence KL

Figure 4: Training reward of ReFT, evaluation accuracy, KL against training epoch on GSM8K P-CoT.

Model Setting	Accuracy
CodeLLAMA + ReFT	75.28
– remove partial reward	74.40
– KL coefficient $\beta = 0$	*collapse*
– non-shared value model	75.15

Table 6: Ablation study on GSM8K P-CoT.

with a small language model. We conduct an experiment on P-CoT data using Galactica-125M[10], Codeparrot-small[11] and Codegen-350M[12]. Table 5 shows the performance comparison between SFT and ReFT. Surprisingly, ReFT still outperforms SFT on three datasets. Such improvements demonstrate the robustness of ReFT during the exploration of reasonable programs.

Ablation Study We perform the ablation study using CodeLLAMA on GSM8K P-CoT (Table 6). Without the partial reward, ReFT obtains a lower accuracy 74.4 but it is still much better than SFT. As mentioned in §3.1, such a partial reward can help reduce the effect of sparse reward (Trott et al., 2019) during training. In addition, the policy distribution will easily collapse to produce unexpected results (i.e., 0 accuracy) if we set the KL coefficient β to 0. It is certainly critical to impose constraints on the space that the policy explores (Ouyang et al., 2022). The initial warm-up step essentially makes such constraints and allows the policy to further explore within the range that is governed by β. We also experiment with a separate value model (Andrychowicz et al., 2021; Cobbe et al., 2021b), where the torso parameters are initialized the same as the policy model. We found that such a setting allows the policy to converge faster in early RL training, but eventually reaches an on par performance. Compared to the original setting of a shared value model, it is, however, twice the computation overhead due to one extra forward-pass, as well as twice the memory cost due to the storage of the separate value net. Finally, in Appendix C we give a case study to show how the generated P-CoT evolve for SFT and ReFT.

[10] huggingface.co/facebook/galactica-125m
[11] huggingface.co/codeparrot/codeparrot-small
[12] huggingface.co/Salesforce/codegen-350M-mono

5 Analysis

Generalization Figure 4 shows the mean reward, evaluation accuracy, and the KL divergence during training of ReFT[13] on GSM8K P-CoT using CodeLLAMA as foundation model. SFT converges and becomes overfiting when approaching 40th epoch. However, we can see the mean reward is around 80% to 90% for the ReFT policy at 40th epoch, and the value accuracy is also increasing. In addition, we can see that the KL divergence (Figure 4 (c)) is very large in the beginning and then maintains a reasonable value between 0 and 10. The stable KL divergence indicates our policy performs exploration within a space that contains appropriate programs. The underlying reinforcement learning mechanism greatly improves the generalization ability of ReFT (Brown et al., 2020).

Qualitative Evaluation We perform a human evaluation to qualitatively assess the output from the SFT model, Warmup checkpoint, and ReFT model. The evaluation uses 50 questions and samples the solutions in GSM8K test set that can be solved correctly by all three models. We ask four different annotators to score the reasoning path according to the following criteria, each scored on a scale from 0 to 1.

- *Logic*: evaluates if the logic leading to the answer is correct.
- *Naming*: evaluates if the variable conveys appropriate and reasonable semantics
- *Compactness*: evaluates if the reasoning paths contain redundant information.

[13] For illustration purpose, we only shows the mean reward and KL for 60 epochs.

Method	Logic	Naming	Compactness	Overall Score
SFT	0.986	0.988	0.994	2.967
Warmup	0.949	0.982	0.990	2.920
ReFT	0.992	0.990	0.996	**2.982**

Table 7: Qualitative scores of models from three methods trained on GSM8k P-CoT dataset.

A perfect score of 3 indicates good performance across these three dimensions. To ensure the evaluation is impartial and faithful, we strictly follow the setting: (1) The origin of each reasoning path (from SFT, Warmup, or ReFT) is anonymized to prevent annotator bias. (2) Four different annotators are responsible for different portions of the samples.

As seen in table 7, though the overall scores are quite close, ReFT performs slightly better than SFT, and outperforms the Warmup variant. Note that SFT is inherently trained to learn from the ground truth, thus, it is likely to have a high score. This comparative analysis underscores the robustness of ReFT in generating accurate and semantically coherent reasoning paths.

When ReFT surpasses SFT? To further investigate the relationship between ReFT and SFT, we perform ReFT training with different number of warm-up steps from SFT. Figure 5 shows the value accuracy of different ReFT variants against SFT[14]. Specifically, if the warmup step is 3, that means the policy initialize from the 3rd-epoch SFT checkpoint. We can see that the performance of all ReFT policies decreases right after the warm-up in the beginning, until the training epoch reaches around 8. Because the linear layer in the shared value model is randomly initialized, and it could take a few epochs to adjust the distribution. Starting from the 30th epoch, SFT converges and all ReFT variants are still improving. We can also see that all variants outperform SFT by a significant margin and there is no obvious advantage of any specific ReFT variant.

6 Conclusion

We have introduced reinforced fine-tuning (ReFT) as a new method for fine-tuning models to solve math problems. In contrast to SFT, ReFT optimizes a non-differentiable objective by exploring multiple CoT annotations in the search for the correct answer, rather than relying on a single annotation.

[14]We only show 60 epochs for illustration purposes. The performance for the later epoch is shown in Figure 4 (b).

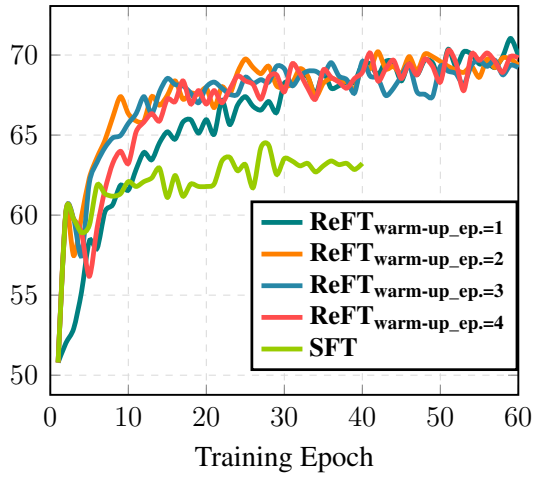


Figure 5: Accuracy comparison between SFT and ReFT with different number of warm-up epoch.

Through extensive experimentation on three datasets using two foundation models, we have demonstrated that ReFT outperforms SFT in terms of performance and generalization ability. Moreover, we have showcased the compatibility of models trained with ReFT with techniques such as majority voting (Wang et al., 2023b) and reward model reranking (Cobbe et al., 2021a; Uesato et al., 2022).

Furthermore, ReFT has exhibited superior performance compared to several publicly available open-source models of comparable sizes in math problem-solving. This demonstrates the effectiveness and practical value of the ReFT approach.

7 Future Work

We have made the first attempt of applying reinforcement learning, specifically the PPO algorithm (Schulman et al., 2017), to fine-tune of LLMs for math problem-solving. Our future work includes utilization of offline reinforcement learning techniques (Levine et al., 2020; Gulcehre et al., 2023), development of a *warm-up free* method to enhance training efficiency and performance, thereby reducing the gap with the reranking method. Additionally, Lightman et al. (2023) suggests that a well-trained process-based reward model (PRM) can significantly enhance performance. Hence, it would be worthwhile to explore the implementation of process-based rewards in reinforcement learning training. Lastly, as ReFT is a versatile approach, we intend to apply it to more general reasoning tasks where the inference can be formalized with CoT.

Limitations

Training Efficiency As depicted in Figure 4 (b), it is evident that ReFT necessitates a greater number of epochs to reach convergence compared to SFT. This is primarily due to the fact that ReFT optimizes a non-differentiable objective and requires exploration of the generation space to attain correct answers. While a larger learning rate may expedite convergence, it also makes the policy more susceptible to instability and potential collapse. Alternatively, using a larger batch size is a viable option; however, it comes at the expense of increased computational costs.

Reward Hacking Our reward function relies solely on the final answer to determine the reward. However, as demonstrated in the experiments conducted on the $\text{MathQA}_{\text{MCQ}}$ N-CoT dataset, the policy can be easily manipulated if the possible space of final answers is limited, such as `A,B,C,D`. To mitigate the issue of reward hacking, it may be necessary to employ a more detailed or process-based reward function that takes into account a broader range of factors.

References

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of NAACL*.

Massih-Reza Amini, Vasilii Feofanov, Loic Pauletto, Emilie Devijver, and Yury Maximov. 2022. Self-training: A survey. *arXiv preprint arXiv:2202.12040*.

Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. 2021. What matters in on-policy reinforcement learning? a large-scale empirical study. In *Proceedings of ICLR*.

Thomas Anthony, Zheng Tian, and David Barber. 2017. Thinking fast and slow with deep learning and tree search. In *Proceedings of NeurIPS*.

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. 2023. A general theoretical paradigm to understand learning from human preferences. *arXiv preprint arXiv:2310.12036*.

Daniel S Brown, Wonjoon Goo, and Scott Niekum. 2020. Better-than-demonstrator imitation learning via automatically-ranked demonstrations. In *Proceedings of Conference on Robot Learning*, pages 330–359.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021a. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Karl W Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. 2021b. Phasic policy gradient. In *Proceedings of ICML*.

Kawin Ethayarajh, Winnie Xu, Dan Jurafsky, and Douwe Kiela. 2023. Human-centered loss functions (halos). Technical report, Contextual AI.

Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. 2021. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese*, 198(Suppl 27):6435–6467.

Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. 2023. Complexity-based prompting for multi-step reasoning. In *Proceedings of ICLR*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of ICML*.

Yang Gao, Huazhe Xu, Ji Lin, Fisher Yu, Sergey Levine, and Trevor Darrell. 2018. Reinforcement learning from imperfect demonstrations. *arXiv preprint arXiv:1802.05313*.

GemmaTeam. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. `https://github.com/huggingface/accelerate`.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. 2023. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*.

Junxian He, Jiatao Gu, Jiajun Shen, and Marc'Aurelio Ranzato. 2020. Revisiting self-training for neural sequence generation. In *Proceedings of ICLR*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. In *Proceedings of Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Steven CH Hoi, Doyen Sahoo, Jing Lu, and Peilin Zhao. 2021. Online learning: A comprehensive survey. *Neurocomputing*, 459:249–289.

Shima Imani, Liang Du, and Harsh Shrivastava. 2023. Mathprompter: Mathematical reasoning using large language models. *arXiv preprint arXiv:2303.05398*.

Zhanming Jie and Wei Lu. 2023. Leveraging training data in few-shot prompting for numerical reasoning.

Zhanming Jie, Trung Quoc Luong, Xinbo Zhang, Xiaoran Jin, and Hang Li. 2023. Design of chain-of-thought in math problem solving. *arXiv preprint arXiv:2309.11054*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of ICLR*.

Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics*, 22(1):79–86.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. In *Proceedings of NeurIPS*.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. 2023. Tinygsm: achieving> 80% on gsm8k with small language models. *arXiv preprint arXiv:2312.09241*.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.

Xiuyuan Lu, Benjamin Van Roy, Vikranth Dwaracherla, Morteza Ibrahimi, Ian Osband, Zheng Wen, et al. 2023. Reinforcement learning, bit by bit. *Foundations and Trends® in Machine Learning*, 16(6):733–865.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*.

Ning Miao, Yee Whye Teh, and Tom Rainforth. 2023. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. *arXiv preprint arXiv:2308.00436*.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. 2021. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*.

OpenAI. 2023. GPT-4 technical report.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Proceedings of NeurIPS*.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are nlp models really able to solve simple math word problems? In *Proceedings of NAACL*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Proceedings of NeurIPS*.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of SIGKDD*.

Martin Riedmiller, Roland Hafner, Thomas Lampe, Michael Neunert, Jonas Degrave, Tom Wiele, Vlad Mnih, Nicolas Heess, and Jost Tobias Springenberg. 2018. Learning by playing solving sparse reward tasks from scratch. In *Proceedings of ICML*.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2018. High-dimensional continuous control using generalized advantage estimation.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. 2023. Beyond human data: Scaling self-training for problem-solving with language models.

Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and characterizing reward gaming. In *Proceedings of NeurIPS*.

Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.

Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. *arXiv preprint arXiv:2211.09085*.

Alexander Trott, Stephan Zheng, Caiming Xiong, and Richard Socher. 2019. Keeping your distance: Solving sparse reward tasks using self-balancing shaped rewards. In *Proceedings of NeurIPS*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. 2020. Trl: Transformer reinforcement learning. `https://github.com/huggingface/trl`.

Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2023a. Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning. *arXiv preprint arXiv:2310.03731*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of ICLR*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of NeurIPS*.

Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. 2020. Self-training with noisy student improves imagenet classification. In *Proceedings of CVPR*, pages 10687–10698.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653*.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D Goodman. 2022. Star: Self-taught reasoner bootstrapping reasoning with reasoning. In *Proceedings of NeurIPS*.

Mengxue Zhang, Zichao Wang, Zhichao Yang, Weiqi Feng, and Andrew Lan. 2023. Interpretable math word problem solution generation via step-by-step planning. In *Proceedings of ACL*.

Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, Limao Xiong, Lu Chen, Zhiheng Xi, Nuo Xu, Wenbin Lai, Minghao Zhu, Cheng Chang, Zhangyue Yin, Rongxiang Weng, Wensen Cheng, Haoran Huang, Tianxiang Sun, Hang Yan, Tao Gui, Qi Zhang, Xipeng Qiu, and Xuanjing Huang. 2023. Secrets of rlhf in large language models part i: Ppo.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.

Aojun Zhou, Ke Wang, Zimu Lu, Weikang Shi, Sichun Luo, Zipeng Qin, Shaoqing Lu, Anya Jia, Linqi Song, Mingjie Zhan, et al. 2023a. Solving challenging math word problems using gpt-4 code interpreter with code-based self-verification. *arXiv preprint arXiv:2308.07921*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. 2023b. Least-to-most prompting enables complex reasoning in large language models. In *Proceedings of ICLR*.

Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*.

Question: Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?

N-CoT: We need to calculate her hourly rate and then multiply it by the amount of time she worked.

First, we need to convert 50 minutes to hours.

There are 60 minutes in an hour, so 50 minutes is equal to 50/60 = 5/6 hours.

Next, we can calculate Weng's earnings by multiplying her hourly rate by the amount of time she worked:

$12/hour x 5/6 hour = $10.

Therefore, Weng earned $10 for 50 minutes of babysitting.

The answer is 10.

P-CoT:

```
def solution():
    """Weng earns $12 an hour for babysitting. Yesterday, she just did
50 minutes of babysitting. How much did she earn?"""
    hourly_rate = 12
    minutes_worked = 50
    hours_worked = minutes_worked / 60
    earnings = hourly_rate * hours_worked
    result = earnings
    return result
```

Figure 6: N-CoT and P-CoT examples on GSM8K

A Examples of N-CoT and P-CoT Representations

We present examples of natural language CoT and program-based CoT from GSM8K dataset in Figure 6. We follow Jie et al. (2023) to perform few-shot prompting and obtain the CoT representations. The natural language CoT is generally the same as the one presented in Wei et al. (2022). The format program-based CoT is similar to the one in PAL (Gao et al., 2023), where we use a function to solve the problems. The function starts with a Python docstring that repeats the question and then a list of statements as reasoning steps.

B Detailed Hyperparameter Setting

Supervised Fine-Tuning We train the model for 40 epochs with the batch size of 48 and the maximum length of 1024.. For small models, we increase the learning rate to $2e$-5, and the number of epoch for training $\text{MathQA}_{\text{MCQ}}$ to 100 epochs.

ReFT Warm-up We perform warm-up for 2 epochs on GSM8K, SVAMP for both N-CoT and P-CoT. For $\text{MathQA}_{\text{MCQ}}$, we perform warm-up for 5 epochs on $\text{MathQA}_{\text{MCQ}}$ N-CoT and 2 epochs on $\text{MathQA}_{\text{MCQ}}$ P-CoT. Specifically for $\text{MathQA}_{\text{numeric}}$, we perform warm-up for 10 epochs because this dataset is much harder and the number of reasoning chains is longer than other datasets. For Galactica-125m and Codegen-350M, the warm-up period is 10 epochs for GSM8K and SVAMP and is 40 epochs for $\text{MathQA}_{\text{MCQ}}$. For Code-parrot, we increases the warm-up period to 40 epochs on all datasets to obtain reasonable warm-up performance.

ReFT RL The maximum length for question is set to 300, and the maximum length during sampling is set to 700. The batch size is 32, which is smaller than SFT due to extra memory consumption of the value model. The number of updates per RL step (i.e., ppo epoch) is set to 2 (Ziegler et al., 2019). We do not employ any weight decay and dropout following Ziegler et al. (2019). For small models, we train for 700 epochs with the learning rate of $3e$-6, the global batch size of 256 and the α of 5, 1 and 0.1 for Galactica-125m, Codeparrot-small and Codegen-350M model respectively.

C Case Study

We show how SFT and ReFT evolve by investigating the generated P-CoT for a specific question. Figure 7 reports the responses of SFT and ReFT at checkpoint epoch 1, 3 and 5. At epoch 1, ReFT is in warmup stage so that its generated P-CoT looks similar to that of SFT. However, later at epoch 3 and 5, the P-CoTs differ. ReFT responses becomes shorter and correct while SFT reponses remains incorrect.

D Attempts with DPO and IPO

In addition to the PPO algorithm in this work, we initially experimented with DPO (Rafailov et al., 2023) and IPO (Azar et al., 2023) in our early attempts on the GSM8K dataset. We obtained the preference data by sampling from the warmup checkpoint and adapted the implementation of these algorithms from the Transformer Reinforcement Learning (TRL) framework (von Werra et al., 2020). Preliminary results indicate that their performance is on par with the "Offline Self-Learning" baseline on GSM8K. This could be explained by the following reasons. Being offline methods, DPO and IPO cannot explore more CoT paths by themselves, thus, their performances are limited by the quality of data sampled from an sub-optimal policy. In addition, their policies are set to maximize the reward given by their implicitly trained reward models. Our preference data, which is sampled from the train set, might not be ideal for reward modeling as there may be neither incorrect responses for

easy questions nor correct responses for hard questions. On the other hand, PPO not only is an online method that explores CoT solution paths automatically, but it also does not require reward model training as the reward function can be directly used. Thus, PPO is chosen in our later experiments.

Example from GSM8K

```
Question:
If Sam and Harry have 100 feet of fence between them, and they agree to split it with Harry
getting 60 feet more than Sam, how much is left over for Sam?
```

SFT

```
# Epoch 1
def solution():
    fence_initial = 100
    fence_difference = 60
    fence_harry = fence_initial + fence_difference
    fence_sam = fence_harry - fence_difference
    fence_left = fence_initial - fence_sam
    result = fence_left
    return result
```

```
# Epoch 3
def solution():
    fence_initial = 100
    harry_feet = 60
    sam_feet = harry_feet - 1
    sam_feet = sam_feet * 2
    sam_feet = sam_feet / 2
    sam_feet = sam_feet
    result = sam_feet
    return result
```

```
# Epoch 5
def solution():
    fence_length = 100
    harry_extra_feet = 60
    harry_feet = fence_length / 2 +
    harry_extra_feet
    sam_feet = fence_length / 2
    left_over_feet = fence_length - harry_feet -
    sam_feet
    result = left_over_feet
    return result
```

ReFT

```
# Epoch 1
def solution():
    fence_total = 100
    fence_harry = fence_total - 60
    fence_sam = fence_harry - 60
    fence_left = fence_total - fence_harry -
    fence_sam
    result = fence_left
    return result
```

```
# Epoch 3
def solution():
    fence_total = 100
    sam_fence = (fence_total / 2) - 60
    result = sam_fence
    return result
```

```
# Epoch 5
def solution():
    fence_total = 100
    sam_fence = (fence_total - 60) / 2
    result = sam_fence
    return result
```

Figure 7: P-CoT responses of SFT and ReFT checkpoints at epoch 1,3 and 5 to the same question in GSM8K dataset. Reponses in green frame are correct while responses in red frame are incorrect.